



PROCESSO SELETIVO AGILIZE · ESTÁGIO EM TECNOLOGIA

Desafio Técnico

Banco — backend (API) + frontend

PRAZO

2 dias corridos · entrega até 00h de 30/06

1. Visão geral

Você vai construir um **Banco**: uma aplicação **fullstack** (backend + frontend). A operação obrigatória é o **saque**; a **transferência** é **opcional** e conta como diferencial. O desafio avalia como você transforma **regras de negócio** em um sistema **funcional, bem organizado e fácil de executar**.

2. Linguagens aceitas

O **backend** deve ser feito em uma destas linguagens:

JavaScript (Node.js)

TypeScript (Node.js)

Python

Ruby

PHP

Go

O **frontend** pode usar HTML/CSS/JavaScript, com ou sem framework (React, Vue, etc.) — fica a seu critério.

3. Arquitetura: backend + frontend

Sua solução **deve** ser dividida em duas partes que se comunicam:

- **Backend (API)**: um serviço que **expõe uma API HTTP** com as operações.
- **Frontend**: uma **interface web** que consome a API e permite ao usuário realizar as operações e visualizar os resultados.

Como você modela o contrato da API, as telas e a forma de preparar as contas fica **a seu critério**.

4. Tipos de conta

O sistema trabalha com **dois tipos de conta**, que se comportam de forma diferente nas mesmas operações:

Característica	Conta Corrente	Conta Poupança
Tarifa por saque/transferência	R\$ 1,00 por operação	Isento
Saldo negativo (cheque especial)	Permitido até R\$ 500,00	Não permitido

5. Operações

Obrigatória:

1. **Saque** — retirar um valor de uma conta, respeitando as regras do tipo de conta.

Opcional (diferencial — conta pontos extras):

1. **Transferência** — mover um valor de uma conta para outra.

6. Regras de negócio

Estas regras serão testadas. Implemente todas:

Regra	Descrição
R1	Conta Corrente: cada saque ou transferência cobra tarifa de R\$ 1,00 além do valor. O saldo pode ficar negativo até o limite de cheque especial de R\$ 500,00 (valor + tarifa não podem ultrapassar esse limite).
R2	Conta Poupança: saques e transferências não podem deixar o saldo negativo e não têm tarifa .

7. Entrega e README

1. Faça um **fork** do repositório base: <https://github.com/agilize/desafio-est-gio>
2. Implemente backend e frontend no seu fork, com **commits ao longo do desenvolvimento**.
3. Envie o **link do seu fork** para calison@agilize.com.br.

O `README.md` deve conter: linguagem e versão do backend; pré-requisitos/dependências; e o **passo a passo exato para subir o backend e o frontend** e usá-los juntos.

⚠ Critério eliminatório: projetos que **não executarem** seguindo o README — ou cujo README não seja suficiente para rodar **backend e frontend** — serão **eliminados**. Teste em uma máquina limpa antes de enviar.

8. Critérios de eliminação

- O projeto não executa conforme o README (backend e/ou frontend).
- Não entrega as duas partes: **backend (API) e frontend**.
- Backend feito em linguagem **fora das aceitas** (seção 2).
- Não atende às regras de negócio das contas (seção 6).
- Cópia ou plágio evidente.

9. O que será avaliado

- **Funcionamento** — backend e frontend rodam, se comunicam e cumprem as regras de negócio.
- **Qualidade do código** — clareza, organização, separação de responsabilidades e ausência de duplicação.
- **Processo** — histórico de commits coerente e um README claro que permita rodar o projeto sem esforço.

Dúvidas: calison@agilize.com.br